

Вступ до Об'єктно-орієнтованого програмування

Заняття 13 (3.1)

C++ для військових технологій

- Парадигми програмування: процедурна vs ООП
- Клас = дані + поведінка. Інкапсуляція (RadioModule)
- Конструктори, деструктори, this
- Наслідування: Device → UAV, GroundStation
- Специфікатори доступу: public, protected, private

Лектор
Євген Гаврилюк

r_d

ПАРАДИГМИ ПРОГРАМУВАННЯ

C++ — мультипарадигменна мова. Ми вже знаємо процедурний стиль. Час познайомитися з ООП.

Процедурний стиль

Дані окремо, функції окремо

```
struct Device { float x, y; };  
void consume(Device& d, float pwr);  
void recharge(Device& d);
```

Проблеми при зростанні коду:

- Хто відповідає за дані?
- Будь-яка функція може зламати будь-яку структуру
- Складно розширювати: кожна нова сутність = нові функції для кожної операції

ООП стиль

Дані + поведінка разом у класі

```
class Device {  
    float x, y;  
    void consume(float pwr);  
    void recharge();  
};
```

Переваги:

- Клас захищає свої дані
- Логіка поруч з даними
- Легко розширювати через наслідування

ЧОТИРИ ПРИНЦИПИ ООП

Класичні стовпи об'єктно-орієнтованого програмування. Сьогодні фокус на 1 і 2.

1 Інкапсуляція

Дані + методи разом.
Доступ контролюється
(private/public).

✓ сьогодні

2 Наслідування

Похідний клас отримує
все від базового.
Розширює та спеціалізує.

✓ сьогодні

3 Поліморфізм

Один інтерфейс —
різна поведінка.
virtual + override.

→ наступні заняття

4 Абстракція

Ховаємо деталі.
Показуємо лише
важливий інтерфейс.

→ наступні заняття

У реальних проєктах ці принципи переплітаються. ООП — не срібна куля, а інструмент для організації складного коду.

КЛАС = STRUCT + МЕТОДИ + КОНТРОЛЬ ДОСТУПУ

У C++ *struct* і *class* — майже однакові. Різниця лише у доступі за замовчуванням.

struct

```
struct Device {  
    Coord position; // public за замовч.  
    float batteryPercent;  
    char callsign[32];  
  
    void consume(float pwr);  
    bool isOperational() const;  
};
```

class

```
class Device {  
private: // за замовч.  
    Coord position;  
    float batteryPercent;  
    char callsign[32];  
public:  
    void consume(float pwr);  
    bool isOperational() const;  
    Coord getPos() const; // getter  
};
```

struct vs class

struct — все public за замовчуванням. Використовуйте для простих даних (Coord, BatteryParams).

class — все private за замовчуванням. Використовуйте коли потрібен контроль доступу (Device, RadioModule).

Конвенція: struct = POD/дані, class = поведінка + інваріанти. Технічно — однакові.

ІНКАПСУЛЯЦІЯ – ЗАХИСТ ДАНИХ

Клас ховає внутрішній стан і дає доступ лише через методи. Це захищає інваріанти.

```
class RadioModule {
private:
    float frequency;    // інваріант: 30.0-300.0 MHz
    float txPower;      // потужність передавача
    int   channel;
    bool  ready;

public:
    void init(float freq, float power, int ch);

    bool setFrequency(float f) {
        if (f < 30.0f || f > 300.0f) return false;
        frequency = f;
        return true;
    }

    void transmit(const char* msg) {
        if (!ready) return;
        // ... передача на frequency MHz
    }

    float getFrequency() const { return frequency; }
    bool  isReady()    const { return ready; }
};
```

Без інкапсуляції

```
radio.frequency = -5;
// Хто завгодно може зламати
// інваріант 30.0-300.0 MHz

radio.ready = true;
// без реальної ініціалізації
```

З інкапсуляцією

```
RadioModule radio;
radio.init(145.5f, 5.0f, 1);
radio.setFrequency(150.0f);
radio.transmit("Bravo-1");

// radio.frequency = -5; ПОМИЛКА!
// private – компілятор не дасть
```

КОНСТРУКТОР І ДЕКТРУКТОР

Конструктор ініціалізує об'єкт при створенні. Деструктор прибирає ресурси при знищенні.

```
class TargetList {
private:
    Coord* data;
    int    count;

public:
    // Конструктор – викликається автоматично
    TargetList(int n) {
        count = n;
        data = new Coord[n]; // виділяємо
    }

    // Деструктор – ~Ім'яКласу()
    ~TargetList() {
        delete[] data; // звільняємо
        data = nullptr;
    }

    Coord& at(int i) { return data[i]; }
    int size() const { return count; }
};
```

Використання

```
{
    TargetList tgts(5);
    tgts.at(0) = {10, 20};
    // ...
} // ~TargetList() тут!
// delete[] автоматично
```

Ключові моменти

Конструктор = ім'я класу
Викликається при створенні об'єкта

Деструктор = ~ім'я класу
Викликається при виході зі scope

**Більше не треба пам'ятати
pro delete[]!**

ВКАЗІВНИК this

Кожен метод неявно отримує вказівник `this` — адресу об'єкта, на якому викликаний.

```
class Device {
    float batteryPercent;
    char callsign[32];
public:
    void setBattery(float batteryPercent) {
        this->batteryPercent = batteryPercent;
        // batteryPercent = параметр
        // this->batteryPercent = поле класу
    }

    float getBattery() const { return batteryPercent; } // OK

    void reset() const {
        batteryPercent = 0; // ПОМИЛКА КОМПІЛЯЦІЇ!
    } // this = const Device* → не можна змінювати
};
```

Коли потрібен `this`

1. Ім'я параметра збігається з полем класу
2. Повернути посилання на самого себе (chaining)
3. Передати себе у функцію

const-методи: `this` → `const Device*`

`reset()` позначений `const` → компілятор вважає `this = const Device*` → запис у поля заборонений.
Правило: якщо метод лише читає — позначайте `const`. Компілятор сам зловить помилку, якщо випадково змінюєте поле.

НАСЛІДУВАННЯ – БАЗОВИЙ КЛАС

Наслідування дозволяє створити новий клас на основі існуючого. Похідний клас отримує все від базового.

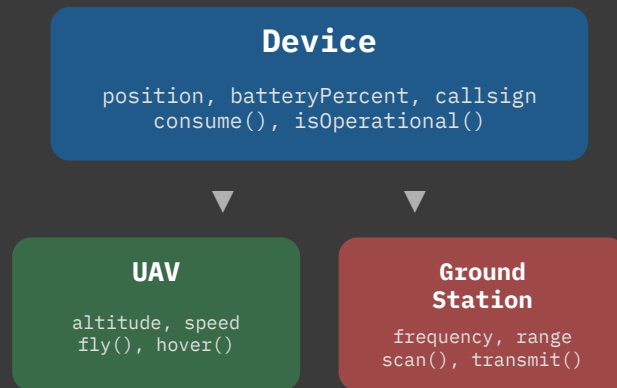
```
// Базовий клас – спільне для всіх пристроїв
class Device {
protected:                // доступно нащадкам
    Coord position;
    float batteryPercent;
    char  callsign[32];

public:
    Device(float x, float y, float bat,
           const char* cs) {
        position = {x, y};
        batteryPercent = bat;
        std::strncpy(callsign, cs, 31);
    }

    void consume(float pwr) {
        batteryPercent -= pwr;
        if (batteryPercent < 0) batteryPercent = 0;
    }

    void recharge() { batteryPercent = 100.0f; }
    bool isOperational() const { return batteryPercent > 0; }
    Coord getPos() const { return position; }
};
```

Ієрархія



protected — поля доступні нащадкам, але не зовнішньому коду

ПОХІДНИЙ КЛАС – РОЗШИРЕННЯ БАЗОВОГО

```
class UAV : public Device {
private:
    float altitude;
    float speed;
    int flightMode; // 0=idle, 1=fly, 2=hover

public:
    // Конструктор викликає конструктор Device
    UAV(float x, float y, float bat,
        const char* cs)
        : Device(x, y, bat, cs) // ← список ініціал.
        , altitude(0), speed(0)
        , flightMode(0)
    {}

    void fly(float dt, float dir) {
        // position доступний (protected)
        position.x += cos(dir) * speed * dt;
        position.y += sin(dir) * speed * dt;
        consume(0.5f * dt);
    }

    void hover() { flightMode = 2; consume(0.1f); }
    float getAltitude() const { return altitude; }
};
```

Використання

```
UAV u(0, 0, 95, "FPV-1");
u.fly(0.1f, 1.57f);
u.consume(5.0f); // від Device!
if (u.isOperational()) {
    // від Device – працює
}
```

Що отримує UAV

Від Device (наслідує):

- position, batteryPercent, callsign
- consume(), isOperational()

Власне (додає):

- altitude, speed, flightMode
- fly(), hover(), getAltitude()

: Device(x,y,bat,cs) — виклик конструктора базового класу

ЛАНЦЮГ КОНСТРУКТОРІВ І ДЕСТРУКТОРІВ

При наслідуванні конструктори викликаються від базового до похідного, деструктори — навпаки.

```
class Device {
public:
    Device(const char* cs) {
        std::cout << "Device(" << cs << ")\n";
    }
    ~Device() {
        std::cout << "~Device\n";
    }
};

class UAV : public Device {
public:
    UAV(const char* cs) : Device(cs) {
        std::cout << "UAV(" << cs << ")\n";
    }
    ~UAV() {
        std::cout << "~UAV\n";
    }
};
```

Порядок виконання

UAV u("FPV-1");

1. Device("FPV-1")

конструктор базового

2. UAV("FPV-1")

конструктор похідного

} // вихід зі scope

Вивід:

```
Device(FPV-1)
UAV(FPV-1)
~UAV
~Device
```

ПРИКЛАД – БАЗОВИЙ КЛАС DEVICE

```
class Device {
protected:
    Coord position;
    float batteryPercent;
    char callsign[32];

public:
    Device(float x, float y, float bat,
           const char* cs) {
        position = {x, y};
        batteryPercent = bat;
        std::strncpy(callsign, cs, 31);
    }

    void consume(float pwr) {
        batteryPercent -= pwr;
        if (batteryPercent < 0) batteryPercent = 0;
    }

    void recharge() { batteryPercent = 100.0f; }
    bool isOperational() const {
        return batteryPercent > 0;
    }
    Coord getPos() const { return position; }
};
```

Device — базовий клас

Спільні дані:

- position — координати
- batteryPercent — заряд батареї
- callsign — позивний

Спільна поведінка:

- consume() — витрата заряду
- recharge() — зарядка
- isOperational() — перевірка

protected — нащадки (UAV, GroundStation) мають доступ до position та batteryPercent

*Спільний код не дублюємо!
consume() працює для всіх
похідних класів.*

ПРИКЛАД — ПОХІДНІ КЛАСИ

```
class UAV : public Device {
private:
    float altitude;
    float speed;
    int flightMode; // 0=idle, 1=fly, 2=hover
public:
    UAV(float x, float y, float bat,
        const char* cs)
        : Device(x, y, bat, cs)
        , altitude(0), speed(0)
        , flightMode(0) {}

    void fly(float dt, float dir);
    void hover();
    float getAltitude() const;
};

class GroundStation : public Device {
private:
    float frequency;
    float range;
    float antennaPower;
public:
    GroundStation(float x, float y,
        float freq, float r, const char* cs)
        : Device(x, y, 100, cs)
        , frequency(freq), range(r)
        , antennaPower(50.0f) {}

    void scan();
    void transmit(const char* msg);
};
```

Похідні класи

UAV (безпілотник):

- altitude — висота польоту
- speed — швидкість
- flightMode — режим
- fly() — політ з витратою заряду через consume()
- hover() — зависання

GroundStation (наземна станція):

- frequency — частота зв'язку
- range — дальність
- antennaPower — потужність
- scan() — сканування ефіру
- transmit() — передача

Обидва класи наслідують:

position, batteryPercent, callsign
consume(), isOperational()

Спільний код в Device — не дублюємо!

Q&A





ЗАВЖДИ Є КУДИ
ЗРОСТАТИ